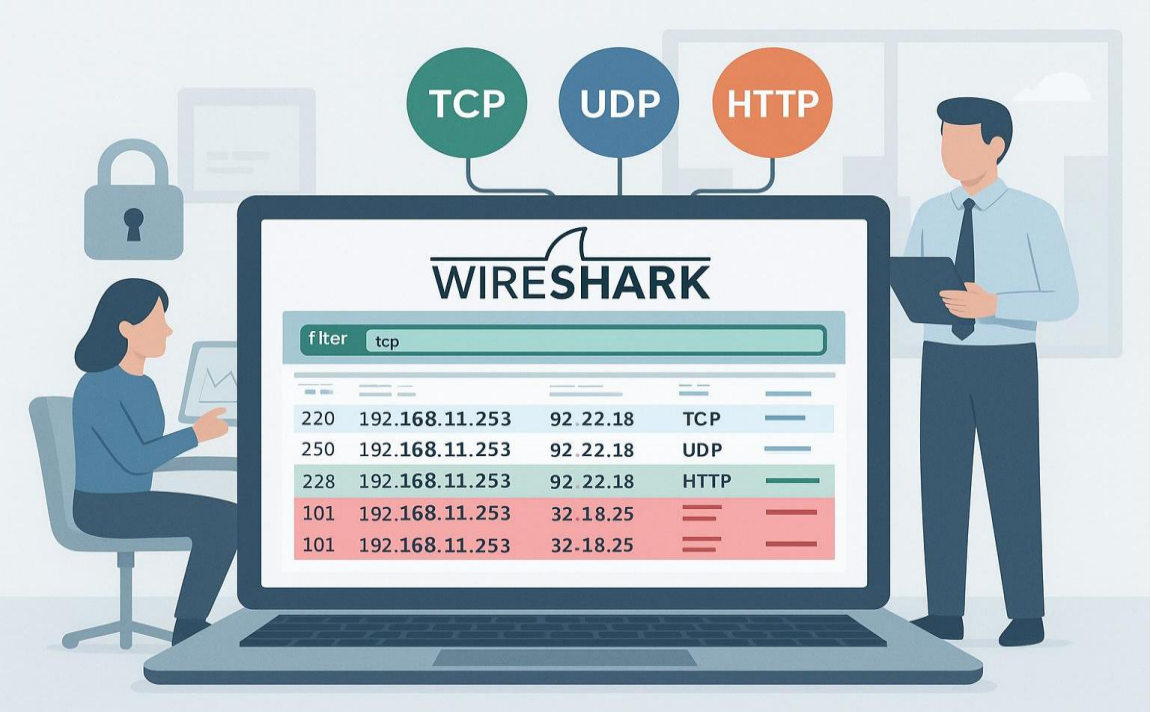
NETWORK TRAFFIC ANALYSIS USING WIRESHARK AND ZEEK (SIMULATION)

A Mini Project Report



Submitted by:

Nikunj Awasthi

2401321720037

B.Tech – Cyber Security

Greater Noida Institute Of Technology

Abstract

- Network traffic analysis is an important part of cybersecurity because it helps identify how devices communicate and detect unusual network activities. This project presents a simulated network traffic analysis system using Wireshark and Zeek.
- A simulated client-server network was created using Linux network namespaces. Network traffic such as HTTP requests and ICMP packets was generated between the simulated client and server. The traffic was captured into a PCAP file using tcpdump and examined using Wireshark. Zeek was then used to analyze the captured traffic and generate structured network logs.
- To automate the analysis, a Python-based analyzer was developed. The analyzer reads Zeek connection logs and classifies traffic as normal or suspicious based on protocol and destination-port rules. In the final experiment, the system successfully analyzed three connections, identifying two as normal and one connection to port 4444 as suspicious.
- The project demonstrates how packet capture, network monitoring, log analysis, and basic automated security detection can be combined into a simple cybersecurity monitoring workflow.

Introduction

- Modern computer networks generate a large amount of traffic every second. Monitoring this traffic is important for understanding network behavior and identifying potentially suspicious activities.
- Network traffic analysis involves examining packets and communication between devices. Tools such as Wireshark provide detailed packet-level information, while Zeek converts network traffic into structured logs that are easier to analyze.
- In this project, instead of using a real physical network, a simulated client-server environment was created using Linux network namespaces. This provides a controlled environment where different types of traffic can be generated safely.
- The captured traffic is analyzed using Wireshark and Zeek. A Python script is then used to automate the analysis and identify unusual destination ports.

Problem Statement

- Traditional manual network monitoring can be difficult because a large amount of network traffic is generated continuously. Security analysts need tools that can capture traffic, extract useful information, and identify potentially suspicious communication.
- The objective of this project is to develop a small simulated network environment that demonstrates how Wireshark and Zeek can be used to capture and analyze network traffic, along with a Python-based mechanism for basic automated detection.

Project Objectives

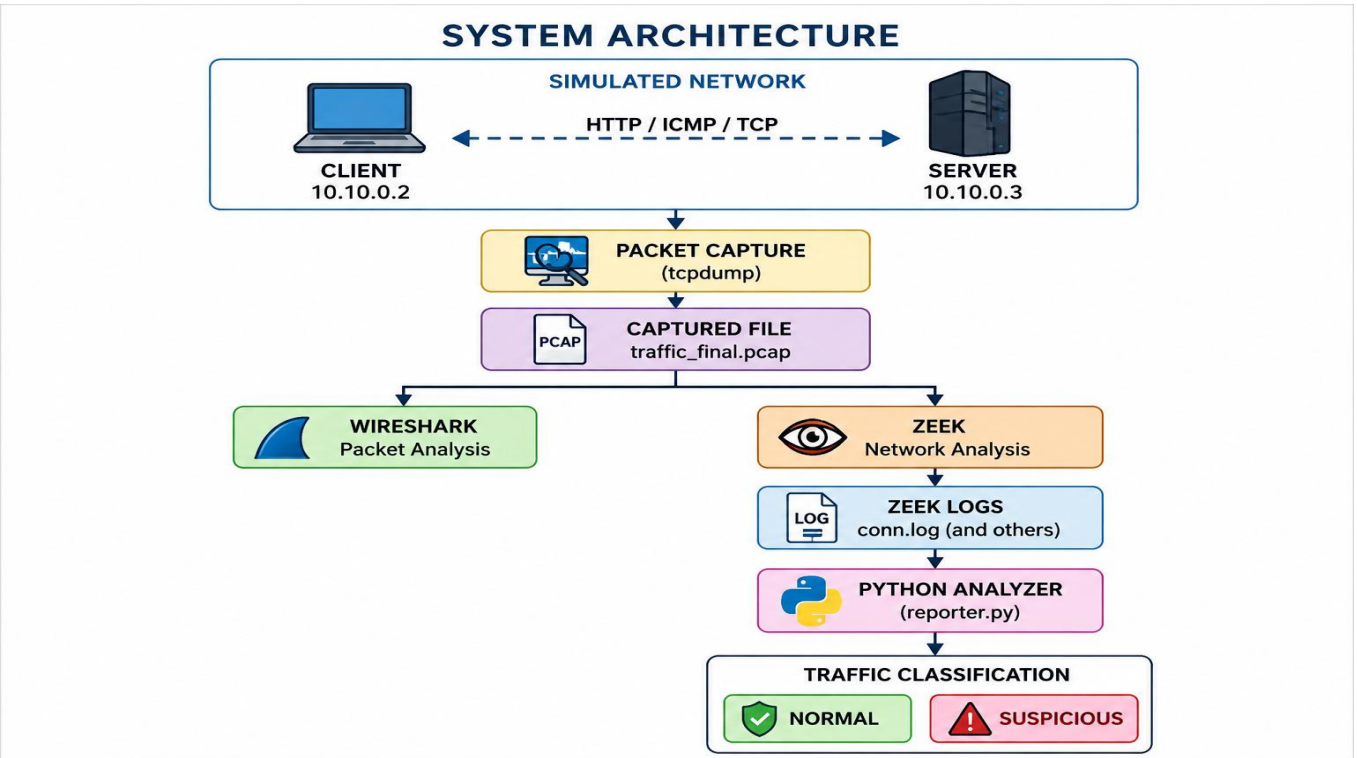
The main objectives are:

1. To create a simulated client-server network.
2. To generate controlled network traffic.
3. To capture network packets in PCAP format.
4. To analyze captured packets using Wireshark.
5. To analyze network traffic using Zeek.
6. To generate structured Zeek logs.
7. To automate basic traffic analysis using Python.
8. To identify unusual or suspicious destination ports.
9. To generate a simple traffic analysis report.

Technologies Used

Technology	Purpose
Kali Linux	Operating environment
Linux Network Namespaces	Simulated network
Python	Traffic generation and analysis
Wireshark	Packet-level analysis
Zeek	Network monitoring and log generation
tcpdump	Packet capture
Podman	Running Zeek container
Bash	Network configuration
PCAP	Captured network traffic

System Architecture



Methodology

Step 1 — Create simulated network

Two Linux network namespaces were created:

- Client → 10.10.0.2
- Server → 10.10.0.3

A virtual Ethernet connection was created between them.

Step 2 — Test connectivity

Connectivity was tested using ICMP:

- ping -c 4 10.10.0.3

The client successfully communicated with the server with **0% packet loss**.

Step 3 — Generate HTTP traffic

A Python HTTP server was started on the simulated server:

- python3 -m http.server 8080

The client accessed the server using:

- curl http://10.10.0.3:8080

Step 4 — Capture network traffic

The traffic was captured using tcpdump:

- tcpdump -i veth-client -w traffic_final.pcap

The resulting PCAP file was later opened in Wireshark.

Step 5 — Analyze with Wireshark

The PCAP file was opened in Wireshark to examine the captured packets.

Filters such as:

- tcp

and:

- tcp.port == 4444

were used to inspect specific traffic.

Step 6 — Analyze using Zeek

Zeek was used to process the PCAP file and generate structured logs.

The main log used in this project was:

- conn.log

This contains information about network connections such as source IP, destination IP, destination port, and protocol.

Step 7 — Automated analysis

A Python script called:

- reporter.py

was developed.

The script reads:

- zeek-logs/conn.log

and classifies connections based on protocol and destination port.

Step 8 — Generate report

The Python script produces:

- reports/traffic_report.txt

The report contains the total number of connections and their classification.

Experimental Results

This is where you put your actual result.

Results

The final experiment produced **3 network connections**.

Category	Count
Total connections	3
Normal connections	2
Suspicious connections	1

The suspicious connection was:

- 10.10.0.2 → 10.10.0.3:4444

The connection was flagged because port 4444 was configured as an unusual/testing port in the Python analysis rules.

The normal traffic consisted of the expected HTTP/ICMP communication in the simulated environment.

Screenshots.

Figure 1 — Simulated Network

Screenshot showing:

- ping -c 4 10.10.0.3

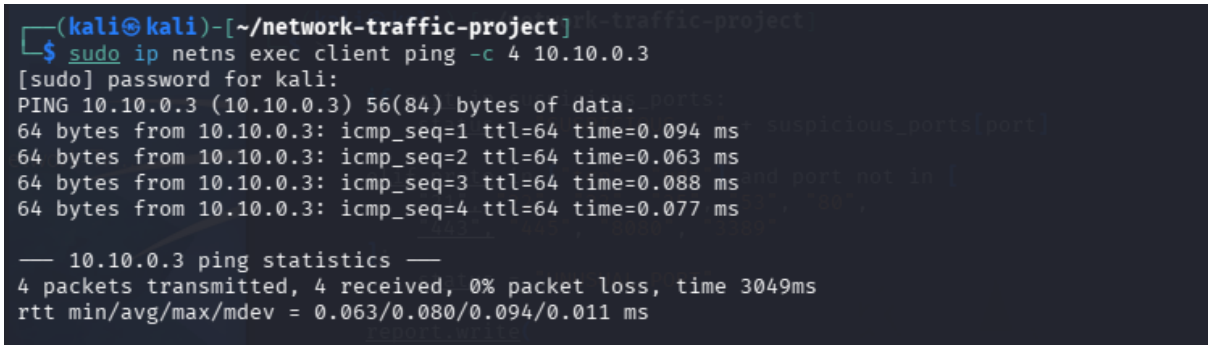


Figure 1: Successful communication between simulated client and server.

Figure 2 — HTTP Traffic

Screenshot showing:

- curl http://10.10.0.3:8080

```
┌──(kali㉿kali)-[~/network-traffic-project]
└─$ sudo ip netns exec client curl http://10.10.0.3:8080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<style type="text/css">
:root {
color-scheme: light dark;
}
</style>
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href="captures/">captures/</a></li>
<li><a href="reports/">reports/</a></li>
<li><a href="scripts/">scripts/</a></li>
<li><a href="zeek-logs/">zeek-logs/</a></li>
</ul>
<hr>
</body>
</html>

┌──(kali㉿kali)-[~/network-traffic-project]
└─$
```

Figure 2: HTTP communication between the simulated client and server.

Figure 3 — Wireshark

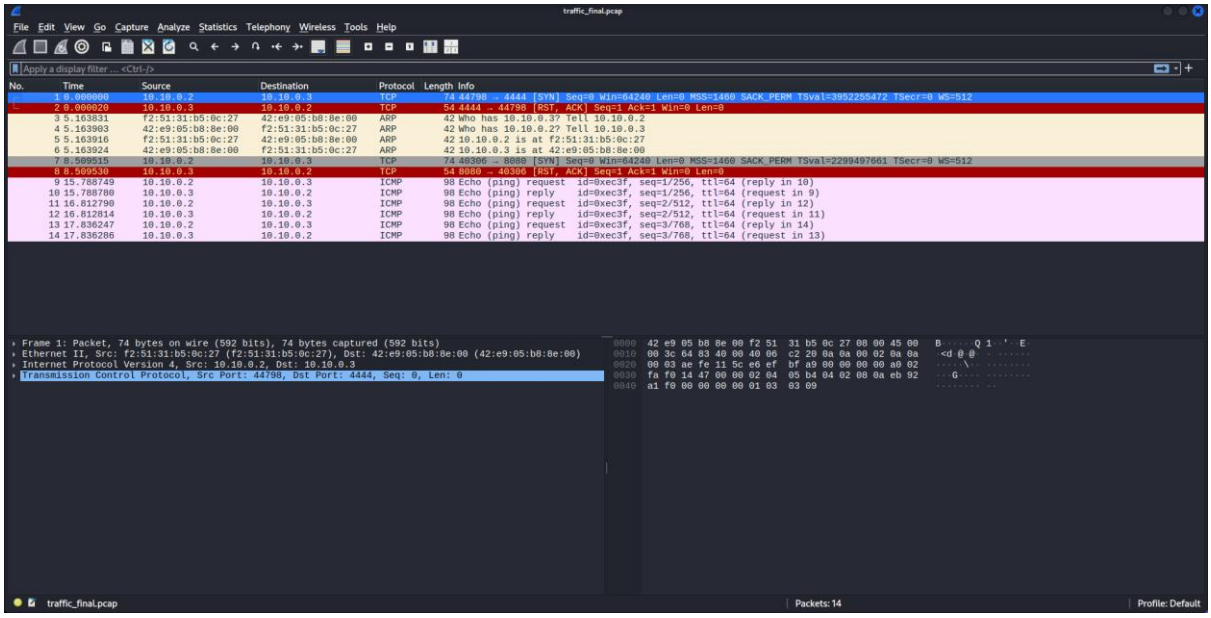


Figure 3: Captured network packets analyzed using Wireshark.

Figure 4 — Suspicious Traffic

Wireshark filter:

- tcp.port == 4444

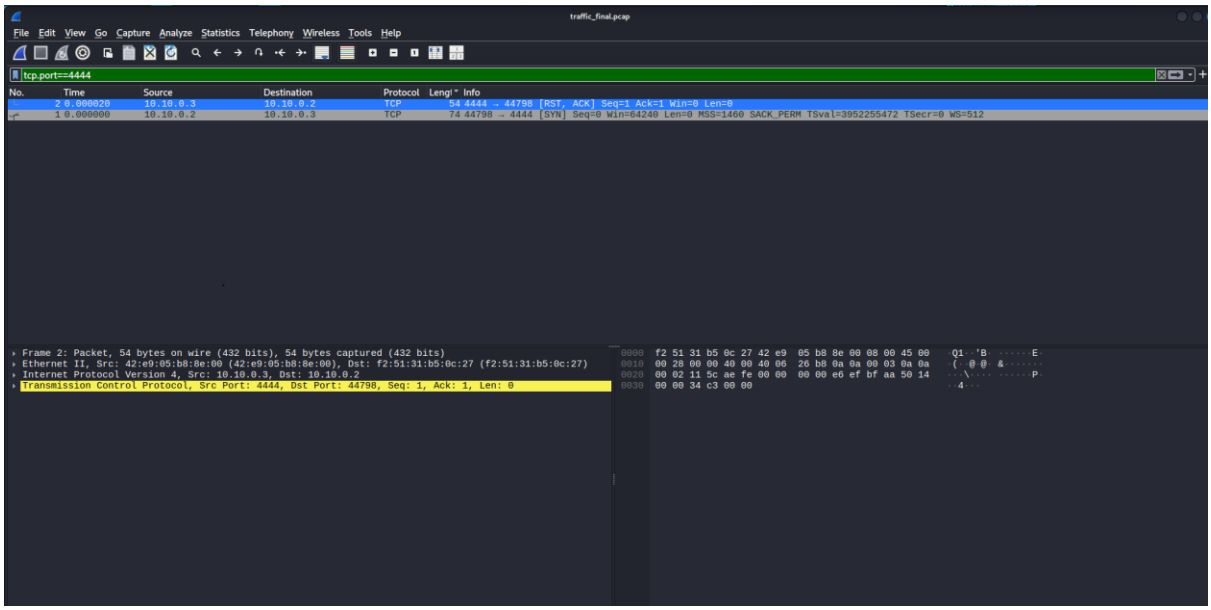


Figure 4: Identification of traffic associated with the unusual destination port 4444.

Figure 5 — Zeek Logs

Figure 5: Logs generated by Zeek from the captured PCAP.

Figure 6 — Automated Analysis

Figure 6: Automated Traffic classification using the python analyzer

Advantages

- Provides a controlled environment for network analysis.
- Does not require physical networking equipment.
- Combines packet-level and connection-level analysis.
- Automates basic traffic classification.
- Helps students understand real cybersecurity monitoring workflows.
- Can be extended with additional detection rules.

Limitations

- The network is simulated rather than a real production network.
- The current detection system uses simple rule-based analysis.
- Only a limited amount of traffic was generated.
- The system does not currently perform advanced intrusion detection.
- Port-based classification can produce false positives.
- Encrypted HTTPS traffic provides less application-level visibility.

Future Scope

The project can be improved by:

1. Adding DNS traffic analysis.
2. Adding more Zeek logs such as DNS and HTTP logs.
3. Implementing machine-learning-based anomaly detection.
4. Adding automatic alert generation.
5. Creating a web-based dashboard.
6. Adding IP reputation checking.
7. Detecting port scanning behavior.

8. Detecting brute-force patterns.
9. Storing results in a database.

Conclusion

- This project successfully demonstrates a simulated network traffic analysis system using Wireshark and Zeek.
- A client-server network was created using Linux network namespaces, and controlled network traffic was generated between the two simulated devices. The traffic was captured as a PCAP file and examined using Wireshark. Zeek was then used to extract structured connection information.
- A Python-based analyzer was developed to automatically classify network connections. In the final experiment, three connections were analyzed, with two classified as normal and one identified as suspicious based on its destination port.
- The project provides a basic but practical demonstration of how network traffic capture, monitoring, log analysis, and automated security detection can be combined in a cybersecurity environment.